

Common Problems

Bot Detection 🎧

Published Jul 2, 2025 · 🍷 medium

Watch Video Walkthrough

Watch the author walk through the problem step-by-step

 Watch Now

Understanding the Problem

Bots are automated malicious actors that can carry out a bunch of activities on any platform (including social media platforms) which hurt user experience and dilute the brand value.

Bots can engage in a wide variety of activities, like creating fake accounts, sending friend requests, sending messages to users, posting duplicate content, posting harmful content or spreading misinformation.

In this system we're going to try to design a bot detection system that can be used to detect bots on a social media platform, with the goal of separating legitimate user activity from automated or malicious bot activity.

Problem Framing

We'll start by establishing a framing for the problem we're looking to solve. There are a lot of different types of bots and detection systems, so zoning in on the problem we're looking to solve will help us focus on the most important aspects.

Clarify the Problem

We'll start by asking targeted questions of the interviewer. These can be questions that just generally help us to understand what the system does, or questions that will dictate later

design decisions in the problem. Expect to spend 3-5 minutes here. More senior candidates will make assumptions (and validate them with their interviewer) whereas more junior candidates will be seeking more clarity from the interviewer.

Some example questions that one might ask:

- What is the base prevalence of bot posts on the platform?
 - *Our interviewer may tell us that, left unremediated, bots make up 50% or more of all actions on the platform but simple heuristics cut this dramatically to <1%.*
- What happens if we wrongly classify a real user as a bot?
 - *We can turn away good actors and see a dip in their engagement. While users can appeal it's expensive to review these appeals and bot creators can navigate the appeals process just as well as legitimate users.*
- How quickly do we need to detect bot activity?
 - *We'll want to catch it as quickly as possible!*
- What is the scale of the problem?
 - *Let's assume we have 500M Daily Active Users, with 1-2 sessions per day*
- Are there any tell-tale signs of bot activity?
 - *Bot authors are constantly adapting to the systems to try to evade detection. In previous iterations it was common for bots to spam requests at a frequency much higher than any human, but the most recent bots are much more sophisticated and are harder to detect.*
- Do we have a golden truth of whether an account is a bot or not?
 - *We have access to expensive investigators who can manually determine the answer, but this is expensive and time consuming, they can investigate low 100s of accounts per week.*

Some things to note here:

- This is an **adversarial** problem. Bot authors are constantly adapting to the systems to try to evade detection.
- Prevalence is high and our bot detection system will be at the front of a lot of user interaction surfaces. Compute will be a key constraint.
- Labelled data is scarce and expensive.



Bot remediation frequently carries a *positive* net impact on compute. That is: the cost of detecting and preventing bot activity consumes less compute than the cost of the bot activity itself. But that doesn't mean we can be inefficient!

We'll make note of the important pieces on our whiteboard for future reference/reminder:

Detecting automated/bot accounts
500M DAU, 1-2 sessions/day
Inhuman behaviors (fast clicks, weird hours)
Low-level of prevalence (<1%)
Investigator labels, low volume and \$\$\$

Problem Clarification

Establish a Business Objective

Next, we need to translate our problem into a business objective. The business objective is distinct from our loss function or ML objective: it's the end-goal the business cares about. We can think of much applied ML engineering as an optimization problem over a business objective.

In the context of bot detection, there are often teams explicitly dedicated to the problem indefinitely. Experienced candidates demonstrate their ability to propose objectives that balance technical feasibility with real business impact.

Bad Solution: Maximize the number of bot accounts detected



Bad Solution: Maximize the accuracy of our model



Good Solution: Maximize bot detection rate, subject to false positive rate constraints ✓

Great Solution: Minimize the impact of bot activity on legitimate user experience, subject to guardrails ✓

We'll proceed with an impact-minimizing objective with guardrails.

Decide on an ML Objective

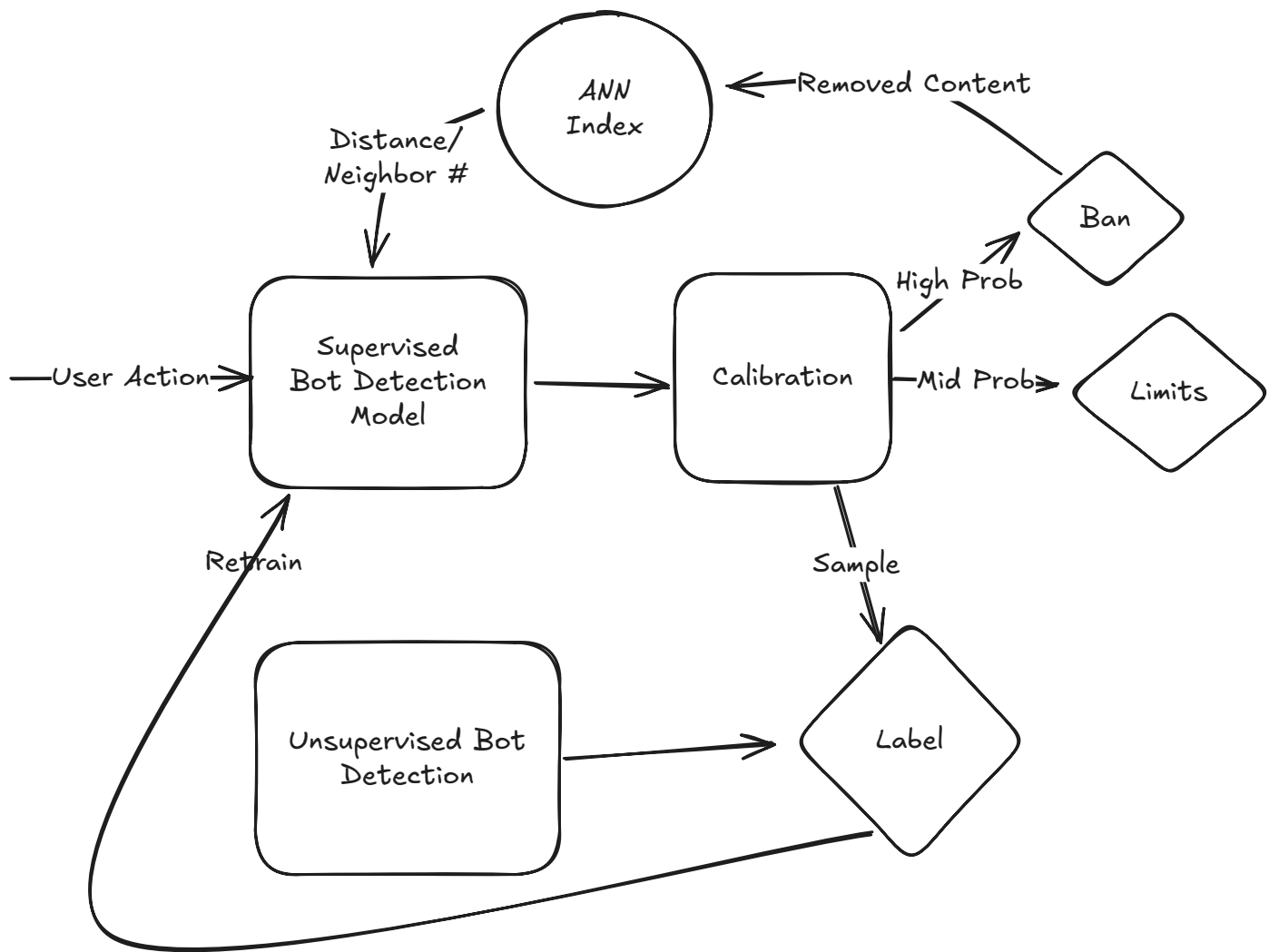
Our ML objective here is straightforward: At heart, the problem we're trying to solve is a binary classification problem. We're trying to classify an account as a bot or not a bot.

High Level Design

For this problem we're going to take two different approaches: for the "obvious" bots or for signatures of bots that we've observed before, we'll use supervised techniques and non-parametric approaches. These will be high precision and help us to ensure that the easy vectors are covered.

For the bots that are harder to detect (or that we haven't detected before) we'll utilize unsupervised techniques.

Since our objective is to minimize the *impact* of bot activity, we'll use a combination of bans, demotions, and limits to achieve our objective. When we have confidence we'll remove the account, but where we may not be sure we'll try to minimize the collateral damage of a false negative by reducing the account's visibility or limiting the volume of interactions the account can have with other users.



High Level Design

The primary goal of this step and diagram is to get a sense for how the pieces of the system fit together rather than finalizing the design before we get into the meat of our modelling.



While non-parametric approaches can be a great optimization for many ML problems (and especially ones that have compute constraints), they're not without their drawbacks. The data we choose to index and include in our nearest neighbor search can have a big impact on the quality of our results. If we have a feedback loop in our system, we risk runaway feedback loops where we gradually index more and more benign data.

It's important that our system distinguishes ground truth labels provided by our investigators from labels generated by our system and weights these accordingly.

Data and Features

Now we'll examine the data sources and features that will power our bot detection models. Since bot authors are constantly adapting their tactics, having diverse data sources is critical for building a robust detection system.

The biggest challenge in an interview setting is clarifying high-level categories of useful data and features without necessarily getting bogged down in the weeds of engineering individual features/signals. Your job is to get a sense for what kind of data your model will need to consume, not necessarily to have dozens of ready-to-implement features.



Bot detection presents unique data challenges compared to other ML problems. The adversarial nature means patterns that work today might fail tomorrow, and getting ground truth labels is expensive and time-consuming. This is a key focal point for the interview!

Training Data

We'll organize our training data into three primary categories, each offering different strengths and limitations:

Ground Truth Labels

Our interviewer mentioned that investigators can manually verify accounts at a rate of hundreds per week. This represents our highest-quality data source, though it's extremely limited in scale. We should strategically use these labels for:

- Validating our model's performance on edge cases
- Creating test sets that reflect real-world bot sophistication
- Calibrating our confidence thresholds

Given the scarcity, we need to be strategic about which accounts we send for manual review focusing on those near decision boundaries (e.g. accounts that are close to the score threshold we choose to separate bots from non-bots) or representing new patterns.

User-Generated Signals

Beyond manual labels, the platform generates numerous signals that correlate with bot activity:

- **Account Reports:** When users flag accounts as suspicious, these provide noisy but scalable labels. We'd expect millions of reports compared to hundreds of manual reviews.
- **Appeal Outcomes:** When restricted accounts successfully appeal, this gives us high-confidence negative labels (accounts incorrectly flagged as bots).
- **Spam/Abuse Reports:** Content-level reports often correlate with the accounts producing that content being bots.

While these signals are noisier than manual reviews, their volume makes them invaluable for training larger models and detecting emerging patterns.

Network-Based Labels

We can leverage the social graph to propagate labels intelligently:

- **IP Address Clustering:** Accounts originating from known malicious IPs (from historical bot campaigns) provide additional positive labels
- **Behavioral Similarity:** Accounts exhibiting nearly identical activity patterns to confirmed bots likely represent coordinated campaigns
- **Registration Patterns:** Bulk account creation from similar sources often indicates bot farms

This approach helps us identify entire bot networks rather than individual accounts, though we must be careful not to propagate errors through the graph.

Synthetic Data Generation

Given the severe class imbalance (bots might represent <1% of accounts in the cleaned dataset), we can use generative approaches to augment our training data. Conditional GANs (like **CALEB**) can generate realistic bot behavior patterns across multiple dimensions:

- Temporal activity patterns
- Content generation styles
- Network formation behaviors

This helps our model learn to detect sophisticated bot patterns that might be underrepresented in our real data.



The adversarial nature of bot detection creates an interesting dynamic: as our detection improves, the bots we observe become more sophisticated (since simple ones are filtered out). This survival bias means our training data naturally shifts toward harder examples over time - both a blessing and a curse!

Our whiteboard might look like this:

Datasets

Labels from investigators
User generated signals
Social graph/network signals
Synthetic/generated data

Important: Balance!

Datasets

Features

With our data sources established, let's design features that capture the multifaceted nature of bot behavior. We'll organize these into logical groupings based on our hypotheses about what distinguishes bots from legitimate users.

Note that candidates outside the trust and safety domain may not be familiar with the full depth of features here. That's ok. Interviewers are generally looking at your instincts and creativity as opposed to your knowledge of a specialized field (unless you happen to be a trust and safety expert!).



Feature engineering for bot detection requires balancing between features that catch current bot behaviors and those robust to adversarial adaptation. Experienced engineers will propose both "easy wins" that catch today's bots and fundamental signals that remain useful as bots evolve.

Activity Patterns

Bots often exhibit activity patterns that differ from human users, even when trying to mimic natural behavior. Ideally these patterns are *learned* by our model, but in many cases expensive sequence models are not feasible to run on every action so hand-engineered features are a good compromise.

- Posting Cadence: Inter-event time distributions (time between posts, comments, likes)
- Click Timing Precision: Variance in reaction times to content
- Error Rates: Typos, corrections, and natural human mistakes
- Session Patterns: Duration and frequency of active sessions
- Circadian Rhythms: Activity distribution across hours/days (humans typically show clear daily patterns)
- Burst Detection: Sudden spikes in activity that exceed human capabilities

These features should be computed over multiple time windows (hourly, daily, weekly) to capture patterns at different scales. We'll use statistical measures like entropy and variance to quantify how "human-like" these patterns appear.

Content Signals

While sophisticated bots increasingly use AI-generated content (i.e. instead of posting the same content over and over), there remain detectable patterns:

- Semantic Diversity: Vocabulary richness and topic consistency across posts
- Duplication Signals: Exact or near-duplicate content (via embedding similarity)
- Language Quality Metrics: Grammar, coherence, and style consistency scores

We'll need to be careful here because not all repetitive or low-quality content comes from bots. These features work best in combination with others.

Instead of exclusively learning these features, we can keep track of bot-posted content and keep it indexed separately in an ANN vector store. Then, when content is posted, we can use the distance or count of nearby content to inform our model. This will help us avoid overfitting to content and reacts more quickly to new patterns than a model which needed to see content examples during training.



Content signals tend to be some of the weakest for bot detection because they're the easiest to fool/fake and often the most visible. If our systems are consistently taking down content that looks like X and not like Y, our adversaries will make more of Y and less of X!

This is less often the case for e.g. temporal activity patterns.

Network Topology Features

The social graph provides rich signals about account legitimacy. We might list a few for our interviewer.

- Follower/Following Ratios: Bots often have skewed ratios
- Network Growth Rate: Speed of connection formation
- Clustering Coefficients: How connected an account's connections are to each other

We can also compute graph embeddings that capture an account's position and role in the network structure.

Account Metadata

Basic account properties provide useful baseline signals:

- Registration Metadata: Account age, verification status, profile completeness
- Authentication Patterns: Login frequency, device diversity, location consistency
- Profile Elements: Username patterns, profile photo characteristics, bio quality

While bots are getting better at faking these, they remain useful especially for catching less sophisticated attacks.

Real-Time Behavioral Signals

As our system operates, we can compute features based on how accounts interact with our detection:

- Detection History: How often this account has been flagged across different models
- Evasion Patterns: Changes in behavior after being flagged
- Appeal Behavior: Frequency and success rate of appeals

This creates a feedback loop where the bot's attempts to evade detection become signals themselves.



A critical consideration: many of these features won't be available immediately when an account is created. We need our model to handle missing data gracefully and potentially use different feature sets for new vs. established accounts. This temporal aspect of feature availability is something many candidates miss!

Summary

Most of this discussion will happen orally, but we may make a few notes on our whiteboard to help us keep track of the features we've discussed. A sample might look like this:

Activity Sequence

- * Time between posts
- * Timing of actions (like clicks, views)
- * Typos/error rates

Content

- * Entropy
- * Distance to previously removed account content

Network

- * Growth rate
- * Connections to removed bot accounts

Real-Time

- * Flags from different models (e.g. spam, pornography)
- * Appeal behavior

Features

Modelling

Production safety/integrity systems are often organically grown in response to real-world attacks. Ideally, we'll indicate this to our interviewer: we're going to start with something simple, see where the gaps are, and then layer on more sophistication where it matters. "Defense in depth" is a security principle that's often applied to bot detection systems with the idea that additional layers provide a safety net.

Benchmark Models

We'll start with a few benchmark models. A simple logistic regression fed with straightforward signals—post frequency, friend-request ratio, login geography, and so on is a good starting point. It trains in minutes, scores accounts in microseconds, and gives us a solid reality check. It's definitely not going to be the end of the road, but it shows we understand the need for a benchmark and can use it to layer on more sophistication where it matters.



A lightweight baseline isn't throw-away work. It becomes the canary that warns when a future release quietly slips in the wrong direction.

Model Selection

With a baseline in place, we pick tools that balance recall, precision, and speed. The biggest mistake candidates make here is assuming that one large content-focused multi-modal LLM is going to solve the problem. Bot detection is more of a challenge of assessing behavior than making determinations of content alone.

Bad Solution: Single, content-heavy mega-model



Good Solution: Graph-sequence approach



By utilizing a model that focuses on graph context (for coordinated operations) and action sequences, we can get a good sense of the behavior of the account.

Model Architecture

The main trunk splits into two specialist branches and then merges their views of each account.

Graph branch – We pull a compact k-hop (usually k=2) graph. We'll use GraphSAGE here because it's **inductive**. Some graph neural networks require us to retrain the model for every new node, but inductive models don't and they scales linearly with the number of sampled neighbors. Relation-specific weights let it treat a "follow" edge differently from a "reply" edge, which keeps spammers who auto-follow thousands of users from looking like well-connected community members. Neighbor sampling caps the fan-out for celebrity accounts so latency doesn't explode. We don't need a deep graph comprehension, two hops is usually enough and saves us from a lot of compute.

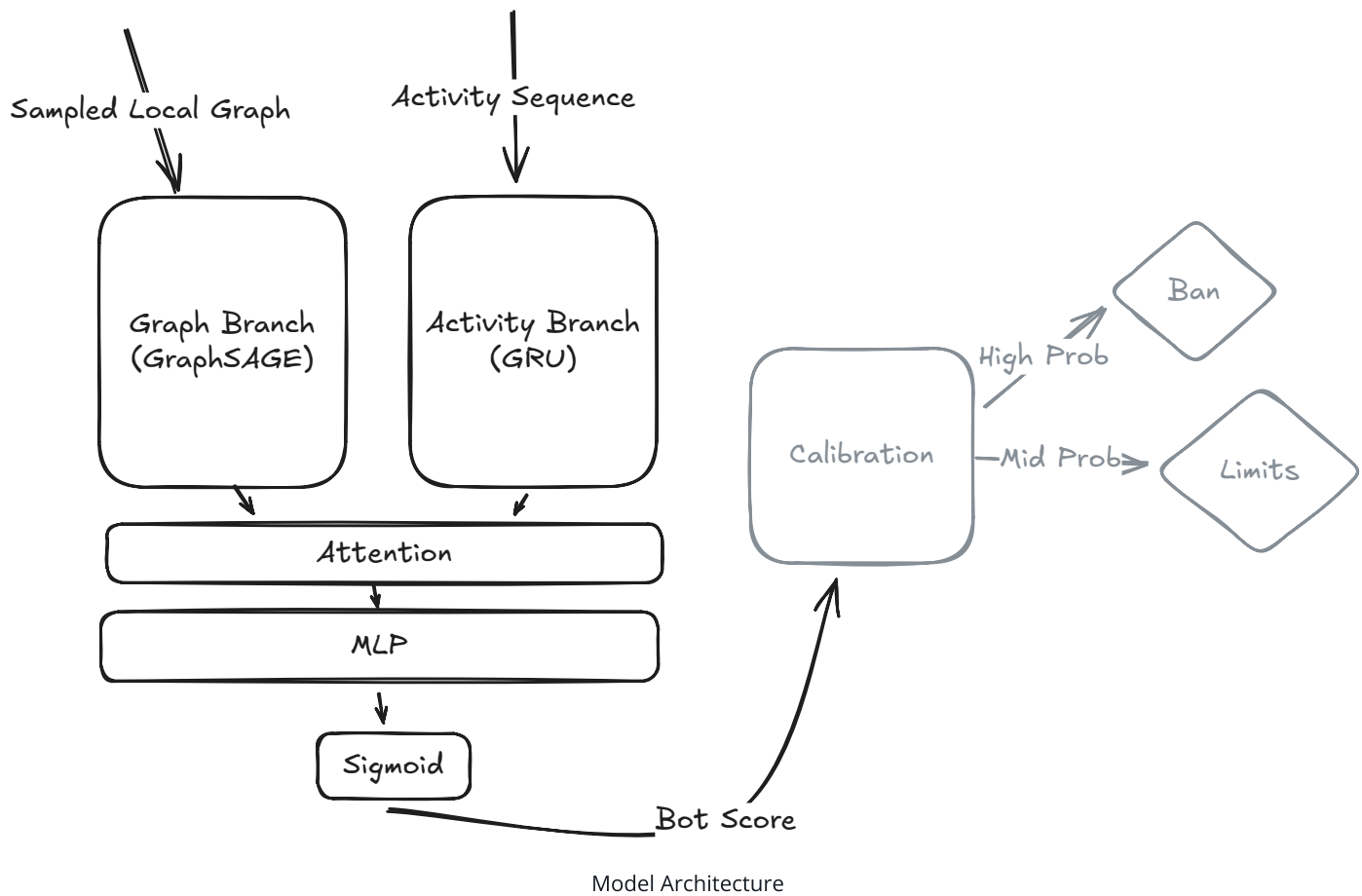
Sequence branch – For our sequence branch we grab roughly the last 200 or so events, bucket timestamps into five-minute slots, embed the event type (post, like, login, device switch, and so on), and feed the sequence through a bidirectional GRU with a small hidden state. We're going to use a GRU here instead of a transformer because we're not trying to capture the full nuance of language, we're trying to fit to sketchy behaviors indicative of bot activity. LinkedIn has a great writeup of using LSTM's for this kind of problem.



As an interviewer, I'm probably not going to penalize you for using transformers here, but you'll get extra points for realizing that a simpler model is more appropriate for this problem.

Fusion – The output vectors of these two branches meet in a single cross-attention layer. That layer lets the timeline query the graph ("does anyone else in my cluster post with this rhythm?") and vice-versa. The attended representation passes through a tiny three-layer MLP (hidden 64 → 32 → 1) that emits a raw risk score.

So our model architecture might look like this:



Pretraining

We're trying to train a rather large model here with limited data and a single binary classification objective. This is a recipe for overfitting and unstable training, so we should look to pretraining to help us capture the color of the data before we fine-tune using our precious investigator labels. Our pretraining will let each half of the model teach itself the "language" of the platform.

For the **graph side**, we take a full snapshot of the social graph and repeatedly hide bits of it by masking node attributes like account age or country, dropping random edges, and shuffling pairs of accounts. In pretraining, the GraphSAGE encoder's job is to guess what we hid and to tell apart real neighbor-pairs from fake ones. In doing so it internalizes the difference between a tight community and a star-shaped follow-farm, learns how likely two accounts are to be connected, and becomes invariant to the odd missing edge.

On the **sequence side**, we stream thirty to sixty days of raw user events into the GRU. We blank out one-in-five tokens, ask the network to reconstruct them, and have it predict the next event in each timeline. Occasionally we split a session in half and make the model recognize

that both clips belong to the same person while treating clips from different users as negative examples. This forces the encoder to absorb human temporal rhythms—sleep gaps, lunch breaks, device swaps—and to notice patterns that feel "too precise" or "too fast" for a person.

Once both branches are fluent on their own, we plug them into the cross-attention fusion head and run a short, label-free alignment pass. Each account now produces two embeddings; the fusion layer is rewarded when those two agree and penalized when they look more like different accounts. By the end of this stage the model can already answer questions such as, "Does anyone else with this network footprint behave with this posting cadence?"

With the heavy lifting done in self-supervision, supervised fine-tuning becomes a light touch: we freeze most layers, expose the model to a few thousand trusted labels, and calibrate the final score so it reads as a true bot probability rather than an arbitrary margin.

Loss Function

Our loss function for this model is a simple weighted binary cross-entropy. We'll use a weight for each training sample that reflects how many real users could have been bothered by that account. We'll want to cap the weights to avoid runaway gradients: even if a bot made a super viral spam post, we don't want our model to be incentivized to over-predict on that.

Inference and Evaluation

Inference System

To get this model into production, we're going to need to handle tremendous scale. Our earlier requirements put the number of actions in the billions which means we're going to be doing a *lot* of inference. We'll need to talk about how we can scale our model to handle this and how we can decide to "investigate" a particular account that might be suspicious.

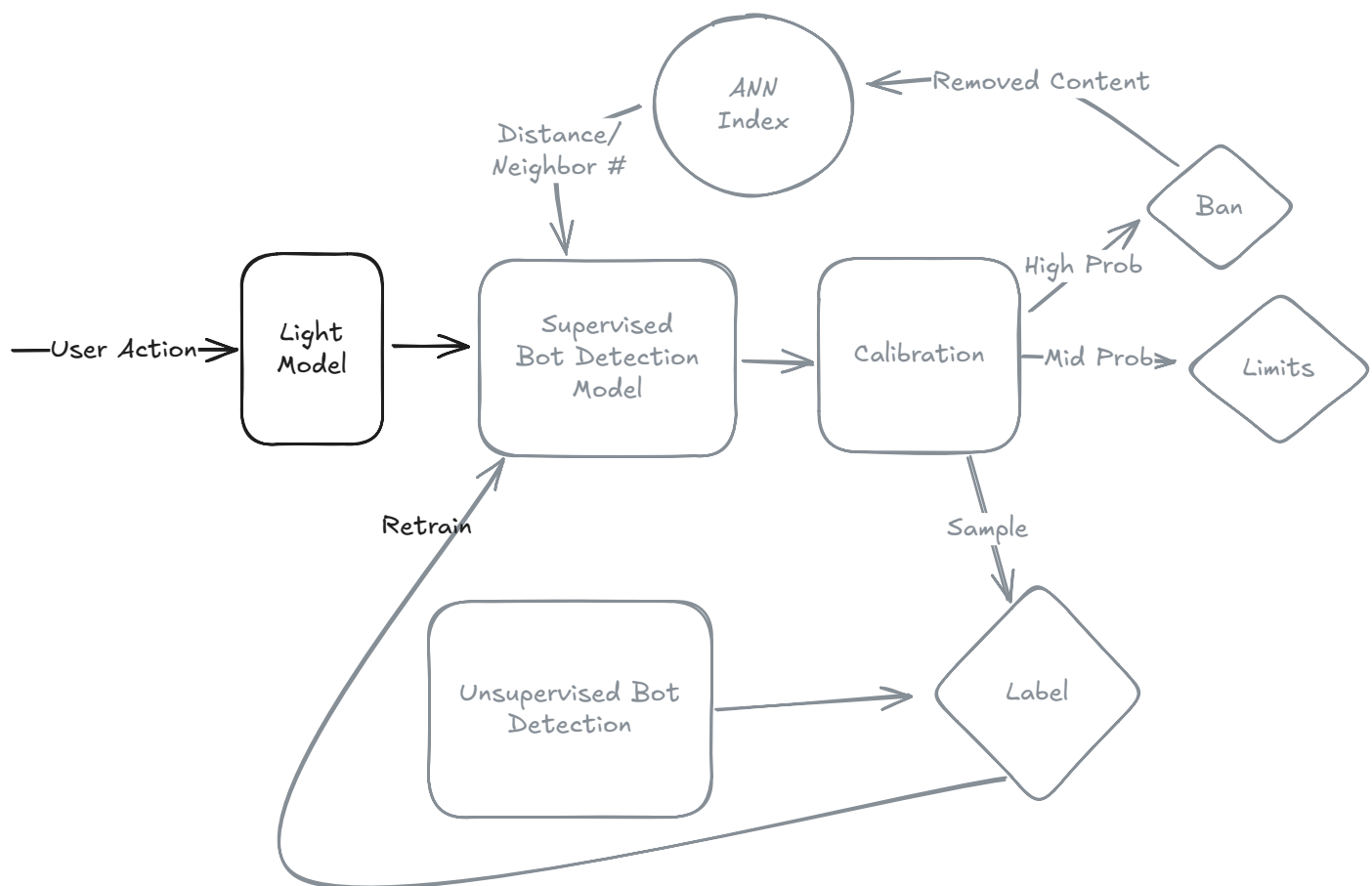
Scaling

Our core, supervised bot detection model is computationally expensive, likely requiring GPUs to run efficiently. To reduce the compute footprint, we can implement several optimization strategies.

First, we can optimize the model. Quantization-aware training allows us to reduce memory requirements while maintaining accuracy. Since many bot accounts exhibit similar patterns, we can leverage caching for our encoders to avoid redundant computations. Account graph embeddings can be stored for a short period of time (potentially invalidated when major shifts happen) and activity pattern encodings can be cached and re-used.

In addition to core model optimization, we can also implement a two-stage architecture to reduce the number of accounts we need to run our expensive model on. This idea is common across most production systems and is a good way to handle scale. Rather than running our expensive graph-sequence model on every account, we implement a two-stage architecture:

1. **Lightweight Filter:** A fast logistic regression model using basic features (posting frequency, account age, device patterns) that quickly identifies obvious legitimate accounts
2. **Heavy Model:** Our full graph-sequence architecture reserved for accounts that pass the initial filter.



Two Stage Architecture

We'll train our lightweight filter in a teacher/student fashion to approximate the output of our more heavy model. This ensures alignment between the two models and serves our direct objective: finding those examples for which the heavy model will score high enough to trigger action.

This cascading approach reduces computational requirements by 80-90% while maintaining detection quality, and it gives us a tunable parameter to adjust resource consumption. If the system is under-resourced, we can dial up the threshold on the lightweight model to reduce the number of accounts we need to run the heavy model on (with some tradeoffs in effectiveness).

Triggering

Our inference system needs to handle both real-time account evaluation and updates based on evolving behavioral patterns. We trigger our lightweight model on several events:

- **Account Registration:** Immediate evaluation of new accounts using metadata and early activity patterns
- **Activity Thresholds:** Re-evaluation when accounts cross suspicious activity boundaries (bulk follows, rapid posting, etc.)
- **Network Changes:** Triggering when accounts join known bot clusters or exhibit coordinated behavior
- **User Reports:** Priority re-evaluation when accounts receive spam/bot reports

Our caching strategy is well-suited for re-triggering since we can quickly update behavioral features without recomputing expensive graph embeddings, provided the account's network neighborhood hasn't changed significantly.

When re-triggering models, we must address several data challenges:

- **Positive Suppression:** Since effective bot detection removes accounts before they exhibit full behavioral patterns, our model risks learning that behavioral data indicates legitimacy
- **Survivorship Bias:** Remaining bot accounts become increasingly sophisticated, requiring constant model adaptation

To handle these we'll need to employ some holdouts. These holdouts not only allow us to collect valuable data, but they obscure the information adversaries might glean from the actions that we take (it's less obvious whether their input triggers the system or not), but we

need to be careful that the holdouts aren't exploited by the bot authors to sneak past detection.

Evaluation Framework

Our evaluation strategy balances two goals: proving new models reduce bot impact on user experience in online experiments, while developing offline metrics that predict online performance.

Online Evaluation

For online evaluation, we run candidate and control models side-by-side, taking action based on only one model's decisions. This requires additional labeling since models flag different account populations.

Given low bot prevalence ($<1\%$), random sampling would require enormous sample sizes. Instead, we use **Importance Sampling** based on model scores:

- Heavily sample accounts near decision boundaries where models disagree
- Down-sample obvious cases (scores near 0.0 or 1.0) with appropriate reweighting
- Focus labeling budget on accounts with high potential user impact

We can leverage proxy metrics for faster iteration:

- **User Reports:** Measuring reports for bot activity, spam, fake accounts
- **Appeal Success Rates:** Tracking legitimate users incorrectly flagged
- **Network Effects:** Monitoring friend request acceptance rates, message response rates

Our primary online metric aligns with our business objective: **reduction in bot interactions per legitimate user** (friend requests received, messages, content views) subject to maintaining $<1\%$ false positive rate on account restrictions.

Offline Evaluation

For offline evaluation, we assess models against held-out test sets using metrics that correlate with online performance:

Primary Metrics:

- **Precision@Recall90:** Aligned with our operational threshold for automated actions
- **PR-AUC:** Stable indicator of overall model discriminative power
- **Impact-Weighted Metrics:** Weighting by potential user exposure (account follower count, activity level)

Diagnostic Metrics:

- **Early Detection Rate:** Fraction of bots caught within first 24 hours of activity
- **Network Coverage:** Percentage of bot clusters detected through graph connections

Fairness Evaluation:

We evaluate model performance across user demographics and geographies to identify potential bias, ensuring our bot detection doesn't disproportionately impact legitimate users from specific communities.

Adversarial Robustness:

We regularly test against simulated bot behaviors and known evasion techniques, maintaining a "red team" dataset of sophisticated bot patterns to ensure our model remains effective as attack methods evolve.

One of the biggest challenges in an adversarial setting is **drift**. We expect the behavior of future bots to be different than the bots we've seen in the past. We need our offline evaluation to reflect this as well as possible. One of the easiest ways to do this is to have the validation sets be stratified by *time*. This way we can ensure that our model performs well on behavior it hasn't (necessarily) trained on.

Deep Dives

Calibration

Our models generate uncalibrated scores, which means the actual actions taken by our model can be wildly different than a naive probability (e.g. 0.6 does not mean 60% of the time the account is a bot). By applying calibration, we can not only adhere to our guardrail metrics we established earlier but we also make the system more stable through migrations and new deployments. We don't want the system to over- or under- enforce when we deploy a new model with a different (maybe better?) score distribution.

There are a few common approaches to calibration:

Good Solution: Histogram Binning



Good Solution: Isotonic Regression



Great Solution: Platt Scaling



Anomaly Detection

We talked earlier about having unsupervised approaches to help us to find the "unknown unknowns" in our data: the bots that are successfully evading our detection. Since bot authors constantly adapt their techniques, we need methods that can spot unusual patterns without relying on labeled examples of every possible attack vector.

Two primary approaches dominate this space: isolation forests and autoencoders. Each has distinct strengths depending on the type of anomalies we're trying to catch.

Good Solution: Isolation Forests



Good Solution: Autoencoders



Great Solution: Ensemble



With our anomaly detection models in place, we can filter accounts that are both (a) anomalous, and (b) causing the harm we care about in our business objectives. We don't necessarily care about accounts that are simply weird, we care about the ones that are weird because they may be bots.

What is expected at each level?

For this problem, mid-level engineers are going to be expected to show their experience building production models with a moderate level of depth. It's important that they come up with a workable solution and have some sensible choices along the way, even if many of their decisions are not the best. Mid-level engineers will set themselves apart by making more optimal decisions, showing better instincts (particularly in data and feature engineering), and being able to spot potential problems.

Senior-level engineers should be able to create a workable solution reasonably quickly which will allow them to deep dive on various aspects of this problem (e.g. the sequence modelling, anomaly detection, etc). They should be familiar with a range of modelling approaches and be able to pick the right one for the problem. Senior-level engineers will spend more time on business objective clarifications, show depth in their reasoning about evaluation, and be able to proactively note where their experience may be insufficient so as to acknowledge where they'd look first if they were tasked with this problem.

Staff-level candidates are going to show a deep understanding of the full scope of the problem (from the business objective to the modelling choices) and be able to make the right tradeoffs. They'll show extensive experience with handling capacity problems and optimizations and be able to address them. They'll have unique ideas for potential directions to improve the system and come up with realistic vulnerabilities and mitigations. Staff-level candidates will typically cover several deep dives of their own, drawing on their experience generalized to this particular problem. They'll also focus more strongly on those things that are most important for the business objective - not just an abstract improvement in accuracy.

Thanks to Shivani Rao for input on this problem breakdown.